

Problem Set 9
COMP301 Fall 2025
28.04.2025 - 02.05.2025

Instructions:

- Submit your answers to the Blackboard PS9 assignment until 03.05.2025 Saturday, at 23.59.
- Submit your PDF file to BlackBoard as *yourIDno_username.pdf*. (Example: *123456_aku-tuk21.pdf*)
- Read the questions carefully. Good luck!

Problem 1: There are 3 parts to this question, you will work on the expression given below. Include your answers in the PDF file.

```
let a = 5 in
  let b = -(a, 1) in
    let a = 3 in
      let c = -(b, a) in
        let func = proc(a) proc(b) -( -(b, a), c) in
          ((func 1) a)
```

Part A. Find lexical depths of all variable references by drawing contour diagrams.

Part. B. Draw the environment diagram when the $-(-(b, a), c)$ is passed to `value-of`. Where in the environment will the variables `a`, `b` and `c` be found for their evaluation?

Note: When drawing the environment, think of the expression as if it was evaluated in PROC language, meaning the environments depicted will not be nameless.

Part. C. Translate the expression into nameless expression.

Problem 2: You have worked on translating programs using lexical addressing, now it is time to reconstruct the named language from the translated version. In this question, you are given the translated versions of PROC program into the nameless language. There are 3 parts to this question, include your answers in the PDF file.

Hint: For example, the program below is the translation of "`let a = 8 in let f = proc(x) -(x, a) in (f 3)`".

```
(a-program
(nameless-let-exp
(const-exp 8)
(nameless-let-exp
(nameless-proc-exp
(diff-exp (nameless-var-exp 0) (nameless-var-exp 1)))
(call-exp (nameless-var-exp 0) (const-exp 3)))))
```

Part A. Translate the nameless expression below into PROC expression.

```
(a-program
  (nameless-let-exp
    (const-exp 3)
    (nameless-let-exp
      (const-exp 2)
      (nameless-let-exp
        (nameless-proc-exp
          (diff-exp (nameless-var-exp 0) (nameless-var-exp 2)))
        (nameless-let-exp
          (call-exp (nameless-var-exp 0) (nameless-var-exp 1))
          (diff-exp (nameless-var-exp 2) (nameless-var-exp 0)))))))
```

Part B. Translate the nameless expression below into PROC expression.

```
(a-program
  (nameless-let-exp
    (const-exp 5)
    (nameless-let-exp
      (const-exp 3)
      (nameless-let-exp
        (diff-exp (nameless-var-exp 1) (nameless-var-exp 0))
        (nameless-let-exp
          (diff-exp (nameless-var-exp 2) (const-exp 1))
          (nameless-let-exp
            (nameless-proc-exp
              (nameless-let-exp
                (nameless-var-exp 0)
                (diff-exp (nameless-var-exp 2) (nameless-var-exp 0))))
            (diff-exp
              (nameless-var-exp 2)
              (call-exp (nameless-var-exp 0) (nameless-var-exp 2))))))))))
```

Part C. We passed the program in Part B to our nameless interpreter. Draw the nameless environment diagram when the sub-expression `(diff-exp (nameless-var-exp 2) (nameless-var-exp 0))` is passed to `value-of`.

Question 1

(a)

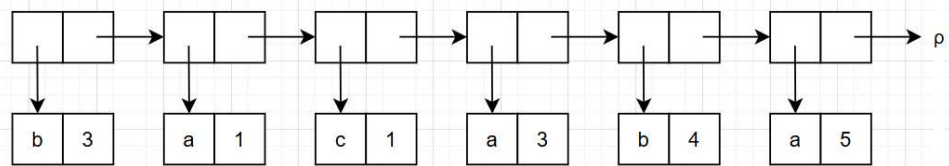
```

let a1 = 5 in
  let b1 = -(a1, 1) in
    let a2 = 3 in
      let c1 = -(b1, a1) in
        let func1 = proc(a2) proc(b2) -( -(b2, a2), c1) in ((func1 1) a2)

```

$-(a, 1) \rightarrow a$ depth 0; $-(b, a) \rightarrow a$ depth 0, b depth 1; $-(-(b, a), c) \rightarrow a$ depth 1, b depth 0, c depth 2; $((func\ 1)\ a) \rightarrow a$ depth 2, $func$ depth 0.

(b)



The lookup will be from left to right, so $a=1$, $b=3$, $c=1$

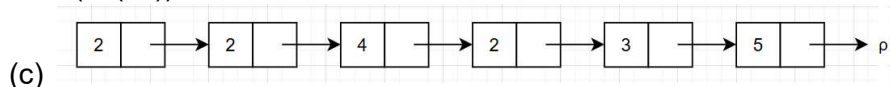
(c) let 5 in

let $-(\#0, 1)$ in

let 3 in

let $-(\#1, \#0)$ inlet proc proc $-(\#0, \#1), \#2)$ in $((\#0\ 1)\ \#2)$

Question 2

(a) let $a = 3$ in let $b = 2$ in let $f = \text{proc}(x) \text{ } -(x, a)$ in let $c = (f\ b)$ in $-(b, c)$ (b) let $a = 5$ in let $b = 3$ in let $c = -(a, b)$ in let $d = -(a, 1)$ in let $f = \text{proc}(x)$ let $a = x$ in $-(d, a)$ in $-(c, (f\ c))$ 

(c)